

Analyzing Performance of Multi-Threaded and Distributed Matrix Scaling Workloads on Cloud Infrastructure

Sri Venkat Bora

25164414

MSc in Cloud Computing

National College of Ireland

Abstract

If a job is compute intensive, the team will have to choose between purchasing a larger machine or many smaller machines, and this decision is typically taken by convention. The literature contains no evidence for either of these answers: shared-memory scaling is known to return well as cores are added, and distributed scaling is known to flatten as workers accumulate, but the two have not been measured on the same workload at the same total core count, so the point at which one outstrips the other remains unknown. The proposed experiment is a controlled experiment that sets the workload, runtime, numerical kernel and instance family to be fixed and only changes the parallelism model, increasing the matrix orders. The completion time and peak memory and per-core utilisation are always recorded together, so it's a measured cross over and not a preference. There is a pilot who determines the number of replications and each order has a confidence interval.

Keywords: parallel scalability, distributed computing, cloud infrastructure, matrix workloads, resource utilisation

1. Introduction

If a compute-bound job exceeds the number of processors in a single machine, then it is possible to increase the number of cores on that machine, or to partition the job onto multiple machines. On cloud infrastructure both are bought, and they both cost differently: A bigger instance costs one unit, while with a cluster it costs network traffic, coordination and idle capacity for every node that completes the job early.

Typically, this decision is made by convention and not by measurement. There is no explicit boundary between a job being "distributed" and one being "threaded," as distribution becomes "threaded" when a job is small and vice versa. The boundary is just what a team needs and just what is missing as neither of the paths scales proportionally.

It is a good candidate to be found in dense matrix operations. They are compute-intensive, have predictable costs as they are traversed in order, they are used in most numerical and machine-learning workloads, and they're supported by well-understood libraries on both paths, making the comparison a test of the parallelism model, not anyone's code.

Before comparisons are fair, one complication must be taken care of. In Python, the interpreter serialises pure-Python threads with a global lock, the compiled numerical kernels that do the numbers release that lock and have their own threads. To measure library thread parallelism, rather than the model under test, the number of threads is fixed here.

Once the kernel is pinned the only question is, what matrix order does distributed execution across multiple cloud instances finish before multi-threaded execution on a single cloud instance with the same total number of cores?

The literature provides two disjointed curves. Sabir and Alebrahim (2025) quantify the shared-memory path, and they find that performance gain is not necessarily linear with doubling cores. Kim, Son and Lee (2022) compare the distributed path on cloud instances and discover comparably sub-linear return, sixty two,5 times slower than the workers buying 2,5 times quicker.

They're both sub-linear, but measured on different hardware with different software at different scales and neither reports the other path as a baseline. It's the decision a practitioner will face and the crossing point has not been seen.

This is made up of four goals. The first compares the performance of both models of parallelism for growing matrix sizes at constant total number of cores. The first compares the performance of both models of parallelism for growing matrix sizes with a fixed number of cores. The second identifies the order in which the crossovers occur or shows that there are no crossovers between the points tested. The third records the peak memory footprint of each model as it is known that memory provisioning can create an apparent performance difference. The fourth is per-core utilisation, exposing capacity billed and not used.

The proposed solution is not a sweep of the benchmarks, but is a controlled experiment. The workload, the runtime of the language, the compiled numerical kernel, the number of instance and the total vCPU count remains fixed and only the parallelism model is changed: threads in one instance vs. workers deployed across different instances summing to the same number of vCPUs. They sweep the matrix and repeat each configuration to give an estimated level of certainty for the crossover point, instead of reading it from a single sweep.

The contribution is a number, not a recommendation: the number of the order in which distribution starts to pay on cloud infrastructure, with memory and utilisation cost on either side of it. The research community gets the first joint measurement of two curves that were only ever observed separately.

Section 2 is a review of what it means to scale out and what it means to scale up, and why they don't add up arithmetically to produce the answer. Workload, configurations, measurements and statistical treatment, ethical position and the schedule are outlined in Section 3.

2. Literature Review

A. *The Limits of Scaling Up*

Sabir and Alebrahim (2025) have explored coarse-grained multi-threaded LU factorisation for a variety of matrix dimensions and conclude that the simple rule of doubling cores for doubling speed does not hold true for their study, primarily because of synchronisation and communication overheads that increase as the

number of cores increases. They are limited as they do not test the scale-out option and do not run on cloud infrastructure. The omission is the basis of this study.

This deficiency on the other hand is less due to parallelism and more due to the nature of the runtime itself, as described in Ruys et al. (2025). As the ceiling can only be guaranteed for one run, the runtime and kernel are specified instead of defaulted here.

From the diagnostic perspective, Jin et al. (2025) look at the same ceiling, finding the causes of scaling loss in big parallel applications and assign load imbalance, resource contention and inter-process communication as the usual suspects. A scaling ceiling is thus a combination of independent causes, not a fixed ceiling, and their work is too large to be scaled up to the level of this study and they give no threshold for changing strategy.

Scaling up is therefore well understood as sub-linear, and reasonably well explained. The other path is next on the list of what happens.

B. The Limits of Scaling Out

There is a limit to distributed execution. Kim, Son and Lee (2022) model distributed matrix multiplication on cloud instances and they found that the workers returned the results 2.5 times faster, which is sub-linear, as in the threaded example. Buyukkaya, Karsavuran and Aykanat (2024) identify a knee at approximately thirty-two machines, and importantly, note that it does not remain at a specific size, because the knee moves with the data.

Both report a single-instance threaded baseline on equivalent hardware, and there are two separate sub-linear curves in the literature, with no point at which one crosses the other.

The cloud studies do not resolve this issue, but make it more complex. Dancheva, Alonso and Barton (2024) present a cloud penalty for various combinations of interconnect and instance tuning, but not for cloud per se, and preserve more than 90 per cent of the relative speed-up achievable on a dedicated supercomputer when using the cloud with an appropriate tuning. Thus, a crossover cannot be derived from figures from differently set up clusters.

The opposite outcome is the outer boundary of that which is assumed. Thomas and Kumar (2018) compared five distributed linear-algebra systems to single-node R and NumPy using a 32 GB dense matrix, ranging in number of nodes from one to eight on a cloud testbed. Nearly none of them obtained a speedup with ordinary least squares for which the Gramian product is the most dominant cost part and one did not even get one node faster than ordinary least squares, which the authors ascribe to communication cost. They are a different generation of systems and their nodes are not "locked" on cores so this is not the matched-resource comparison that is suggested here. It does, however, make it clear that the benefits of scale-out are not assured, and so a "crossover" is considered below as an expectation, but not a premise.

The choice of framework is another confounding effect that can be confused with a finding. With the same doubling of memory per worker, Dugre, Hayot-Sasson and Glatard (2023) measure a gain of 1.79x to 2.71x between the two distributed Python frameworks, which disappears completely. That is the best argument in this literature for instrumenting peak memory in addition to completion time; otherwise, a provisioning decision is a measure of performance. The other side of the coin is equally clear, according to Kerney et al. (2024) that task granularity, not framework identity, dictates parallel efficiency.

The mechanism is provided by Luan et al. (2023): shuffle and data-movement architecture is the dominant distributed performance approach for redistribution of data across stages, a cost which the smaller orders can't amortise.

The reviewed studies are compared to this proposal in Table 1. Only the columns that are important are the models measured, matched core count and memory instrumented, and all the reviewed rows have all three.

Table 1 Reviewed studies contrasted with the proposed experiment

Study	Threaded measured	Distributed measured	Matched core count	Memory instrumented	What it leaves open
Sabir and Alebrahim (2025)	Yes	No	n/a	No	The scale-out alternative, and cloud
Kim, Son and Lee (2022)	No	Yes, on cloud	n/a	No	No threaded baseline on equal hardware
Buyukkaya et al. (2024)	Hybrid	Yes	No	No	A knee, but not against a threaded curve
Thomas and Kumar (2018)	Single-node R and NumPy	Yes, five systems	No, cores grow with nodes	No	Whether scale-out pays at matched cores
Dancheva et al. (2024)	No	Yes, on cloud	n/a	No	Tuning effects, not a crossover
Dugre et al. (2023)	No	Two frameworks	No	Yes	Frameworks compared, not models
This proposal	Yes	Yes	Yes, equal aggregate vCPUs	Yes, peak resident	Answers the above

C. *Why This Research Is Necessary*

Three of the key things are decided. Scaling is sub-linear with typically less than a doubling of speed for each doubling of cores, for shared-memory scaling (Sabir and Alebrahim, 2025). Distributed scaling is sub-linear too, with a knee that scales with the dataset (Kim, Son and Lee, 2022; Buyukkaya, Karsavuran and Aykanat, 2024). Cloud penalty is a configuration property and not inherent (Dancheva, Alonso and Barton, 2024).

The point where the two curves intersect is not determined. The comparison of both models requires a common workload to be used with matched aggregate core count, the only setting in which a crossover can be seen and not only inferred, and for the curves to be combined arithmetically, one must be subtracted from the other but this would be meaningless if there were configuration differences between the experiments that outweighed the difference to be isolated.

Nor have memories been looked after as a result. When per-worker memory changes, as demonstrated by Dugre et al. (2023) a framework advantage disappears; however, no matrix-scaling study instruments report resident memory in addition to completion time, thus the question remains whether reported differences are performance differences or provisioning artefacts. A team thus cannot determine when and if it is the proper time to make a change in strategy, nor if it would be a true change. This proposal provides for them both.

D. *Research Niche*

There has never been a study in the literature to compare multi-threaded and distributed execution of the same workload for matrix computations, on the same aggregate number of cores, on cloud infrastructure, but with peak memory instrumented. None of the reviewed studies measure more than two of those four: Sabir and Alebrahim (2025) measure threading carefully but report no measure of distribution nor cloud; Kim, Son and Lee (2022) report a measure of distribution for cloud instances but give no measure of threading and

provide no baseline of distribution; Dugre, Hayot-Sasson and Glatard (2023) give a measure of memory but they compare two distributed frameworks and not two parallelism models.

The baseline is by Sabir and Alebrahim (2025). The reason their work is interesting is that sometimes twice the cores is not twice the speed, and their claim that they don't consider the scale-out path is the issue addressed here. This study compares what they left out to what they measured, keeping completion time as the common measure, with core count being the measure they kept.

Three expectations are noted as thresholds so that a measurement can be against them: (1) the distributed model has a crossover point within the range tested, as opposed to one model being superior at all orders, for which Thomas and Kumar (2018) have reason to doubt; (2) the distributed model has lower per-core utilisation below the crossover point, meaning that capacity is billed and unused; and (3) peak memory per worker is the limiting factor at the smaller orders, not completion time.

Practitioners receive a threshold, not a preference, based on the order of the problems. The experiment is detailed in section 3. Other factors that have been deferred to future work include: sparse matrices, accelerator backend, clusters bigger than the small fixed cluster used here, and spot-instance pricing.

3. Research Method & Specification

A controlled experiment answers this question. The same workload is run under two parallelism models at equal total core count, across increasing matrix orders, with everything else held constant. Figure 1 shows the arrangement.

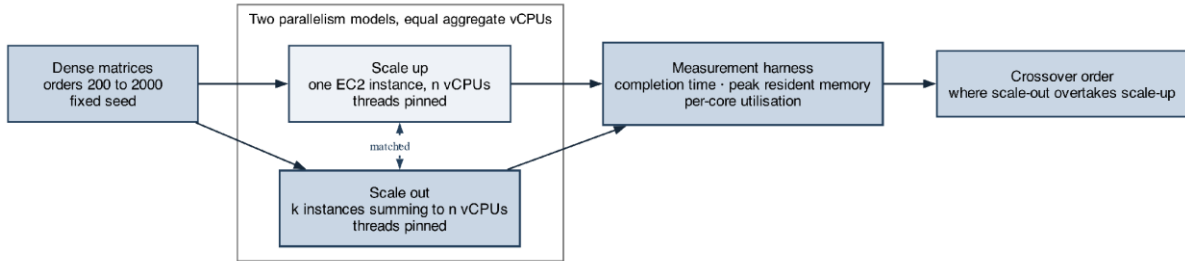


Figure 1 Experimental setup: one workload, two parallelism models at matched core count

A. Research Method

The independent variable is a parallelism model, with two levels: multi-threaded execution within a single cloud instance; distributed execution across a small number of fixed instances, with the same total number of worker vCPs. The order of the matrix is a factor that was varied over six different values ranging from 200 to 2000, for the smallest to fit easily in one worker's memory and the largest to be more dominated by the arithmetic than start-up costs. The workload is a dense matrix multiplication, with known growth cost and well-developed implementations on both.

The dependent variables are completion time, peak resident memory used by all participating processes, and per-core utilisation, the mean fraction of the available vCPUs that is kept in use during the run. The completion time against core count is reported in the same way as Sabir and Alebrahim (2025), so the cloud result can be read on their shared-memory curve.

The level of vCPU is the control that makes the comparison a comparison. If it didn't exist, the study would have been concerned with the amount of hardware purchased, not the way in which it was organised, which is what draws the existing curves away from the ability to be compared to each other. The two conditions are the threaded condition running on a single instance with n vCPUs and the distributed condition running on a cluster with n equal to the sum of workers, both of which have a uniform instance family, so that the clock speed and memory bandwidth per core are equal.

The instance family, total vCPU count, language runtime and its version, the compiled numerical kernel, the matrix generation seed and the region are held constant. There are two confounds that need to be dealt with. The numerical kernel threads internally and unlocks the interpreter lock, therefore an unrestricted library would just run in parallel inside the distributed workers and make their cores show as more than they actually have, kernel thread counts are fixed at runtime and checked at run time. In isolation, Dugre, Hayot-Sasson and Glatard (2023) demonstrate that the impression of a performance difference can be created that isn't due to an actual difference in memory performance per worker; memory per worker is thus assumed to be the same and peak resident memory is reported as a result rather than a parameter.

The measured completion time includes internally the time for start-up and data distribution, as would be paid for by a practitioner and excluding them would be a boost to the distributed condition at just the small orders expected to show a crossover. There is only one instance family, one operation and one small cluster size for external validity. For any given number of cores, the percentage of utilisation sampled is not necessarily the same, but rather an approximation to true occupancy, though sampled at a fixed interval.

B. Research Resources

The experiment needs to use cloud compute instances from one general purpose family, a Python run-time with a fixed compiled numerical kernel, and one distributed execution framework. All configurations are provisioned from infrastructure code and this means that runs are reproducible and that instances are destroyed after every campaign, therefore limiting spend.

The test data is synthetic; that is, dense matrices of the specified orders are generated from a recorded seed, and therefore the same set of data can be used for all the configurations and can be generated again. There is no personal data, no proprietary data and no third party data, no participants are involved. The matrices are generated only once and reused for both conditions; if they were re-generated, this would bring up a difference that is not being measured in the study.

The instrumentation is intentionally simple: completion time from the process boundary; peak resident memory from the operating-system counters sampled across all participating processes; and per-core utilisation from the same counters at a fixed sampling time. Publication of measurement code, infrastructure definitions and collected results.

C. Evaluation

The comparison of the completion time for both models at each order of the matrix, and determination of the order at which the difference in time crosses zero, answers objectives one and two. All three outcomes are from one campaign, as objective three was based on the memory collection during the same runs, and objective four was based on the utilisation sampling.

Before there's any data, each comparison is presented as a null and an alternative. It applies to the completion time, where H_0 denotes no difference in completion time for a fixed matrix order, and H_1 denotes a difference

in completion time for a fixed matrix order. H_0 : There is no difference between memory models at a given order and H_1 : there is a difference. H_0 is the hypothesis that the proportion of the mean of vCPUs used is not different while H_1 is the hypothesis that the proportion of the mean of vCPUs used is different. None is one-sided, since there is no evidence presented to support predicting which model will win at any order.

The selection of the test comes after a normality test, not before. If the distributions are normal, the two models are compared by an independent-samples t-test at each order with Cohen's d as the effect size; if they are not normal, a Mann-Whitney U test is used in place of the independent-samples t-test, with a rank-biserial correlation as effect size. The smallest order for which the distributed model is significantly faster, and continues to be faster for all larger order tests, is referred to as the crossover order; a 95% confidence interval is provided at each order so the crossing is “bracketed” rather than asserted.

All tests are performed at an alpha of 0.05 and Holm-Bonferroni corrected, as each comparison is based on a single campaign. The replication is predetermined and not selected after results are received. The run-to-run variance of each configuration is calculated from a pilot of five runs at the smallest and largest matrix order, and the number of runs to replicate is set at the number of runs required to be 80 per cent confident that a ten per cent difference in completion time for each configuration is not due to a chance error. The smallest difference is also set at 10 per cent, as a difference smaller than that would not be considered practically meaningful because it would not withstand instance-to-instance variability.

D. Ethical Considerations of the Research

No one participates and no personal data is dealt with at any time. For every matrix, experiment is conducted based on a seed that is recorded, and therefore consent and responsibilities for data protection protocols are not part of the study.

Use of a commercial platform for sustained compute-intensive workloads must be within the acceptable-use policy of the provider. All instances are created in the researcher's own account, only execute the workload under study, and are destroyed immediately after each campaign (don't run them idle for later). The size of the clusters is constrained to the small fixed size that the design requires.

The direct costs associated with this study is Compute, the direct environmental impact of this study is the direct environmental impact of the matrix-multiplication campaign, which is proportional to the number of cores used. Run counts are not pushed up to the ceiling as the power allows, but rather depend on the power calculation; matrices are generated once and reused; and the largest order is less than the size pushed up by budget constraints and is bounded by the design. The spend and instance-hours used are included in the results.

Care must be taken in reporting integrity when comparing setups may be skewed. Cluster start-up and data distribution is measured as part of the time instead of being excluded to benefit the distributed model, and per-worker memory is accounted for proportionately, so that the artefact that has been found by Dugre et al. (2023) would not be reproduced without it being noticed. The measurement code and raw result data are published to enable the crossing to be contested.

E. Project Plan

Figure 2 sets out the twelve-week schedule. Weeks one to three provision the instance family, pin the runtime and kernel, and build the measurement harness, ending in a pilot that establishes run-to-run variance and confirms the thread pinning holds.

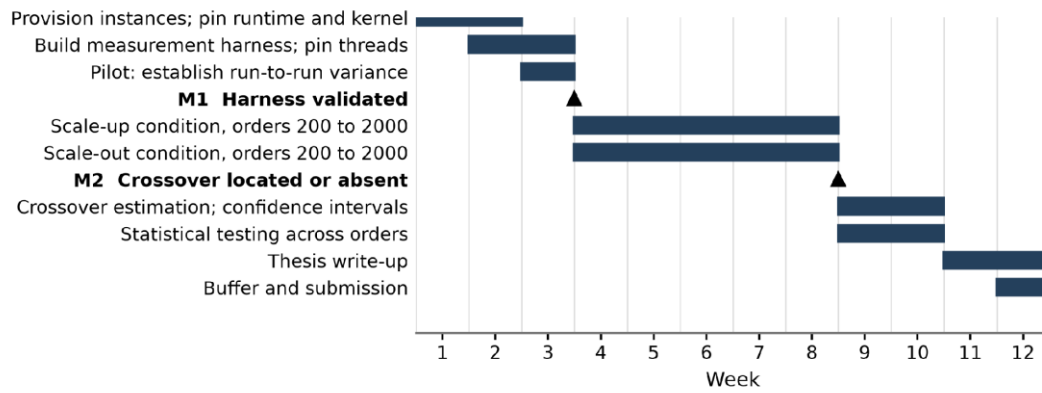


Figure 2 Twelve-week project plan

Then, starting from matrix order in week 4, the conditions of threaded and distributed are run independently, one by one, such that when one of these conditions fails the remaining conditions are not affected. Crossover estimation and statistical testing are in weeks 9-11 and the write-up starts in week 11 and contingency is week 12. The greatest threat is if the variance turns out to be greater than the pilot indicates, and placing the pilot too early is supposed to uncover that.

4. References

- Buyukkaya, K., Karsavuran, M.O. and Aykanat, C. (2024) 'Stochastic gradient descent for matrix completion: hybrid parallelization on shared- and distributed-memory systems', *Knowledge-Based Systems*, 283, 111176. doi: 10.1016/j.knosys.2023.111176. [Journal - JCR Impact Factor 7.2 (2024), Q1]
- Dancheva, T., Alonso, U. and Barton, M. (2024) 'Cloud benchmarking and performance analysis of an HPC application in Amazon EC2', *Cluster Computing*, 27(2), pp. 2273-2290. doi: 10.1007/s10586-023-04060-4. [Journal - JCR Impact Factor 3.6 (2024), Q2]
- Dugre, M., Hayot-Sasson, V. and Glatard, T. (2023) 'Performance comparison of Dask and Apache Spark on HPC systems for neuroimaging', *Concurrency and Computation: Practice and Experience*, 35(21), e7635. doi: 10.1002/cpe.7635. [Journal - JCR Impact Factor 1.5 (2024), Q3]
- Jin, Y., Wang, H., Tang, Q., Guo, X., Zhao, J., Hoefler, T., Liu, T., Liu, X. and Zhai, J. (2025) 'Leveraging graph analysis to pinpoint root causes of scalability issues for parallel applications', *IEEE Transactions on Parallel and Distributed Systems*, 36(2), pp. 308-325. doi: 10.1109/TPDS.2024.3485789. [Journal - JCR Impact Factor 5.6 (2024), Q1]
- Kerney, J., Raicu, I., Raicu, J. and Chard, K. (2024) 'Towards fine-grained parallelism in parallel and distributed Python libraries', in *Proceedings of the 2024 IEEE International Parallel and Distributed Processing Symposium Workshops*. San Francisco: IEEE, pp. 706-715. doi: 10.1109/IPDPSW63119.2024.00133. [Conference - CORE Ranking A (ICORE2026, IPDPS); workshop track]
- Kim, J., Son, M. and Lee, K. (2022) 'MPEC: distributed matrix multiplication performance modeling on a scale-out cloud environment for data mining jobs', *IEEE Transactions on Cloud Computing*, 10(1), pp. 521-538. doi: 10.1109/TCC.2019.2950400. [Journal - JCR Impact Factor 5.3 (2024), Q1]
- Luan, F.S., Wang, S., Yagati, S., Kim, S., Lien, K., Ong, I., Hong, T., Cho, S., Liang, E. and Stoica, I. (2023) 'Exoshuffle: an extensible shuffle architecture', in *Proceedings of the ACM SIGCOMM 2023 Conference*. New York: ACM, pp. 564-577. doi: 10.1145/3603269.3604848. [Conference - CORE Ranking A* (ICORE2026)]

- Ruys, W., Lee, H., You, B., Talati, D., Park, J., Almgren-Bell, J., Yan, Y., Fernando, M., Erez, M., Gligoric, M., Burtcher, M., Rossbach, C.J., Pingali, K. and Biros, G. (2025) 'Performance characterization of Python runtimes for multi-device task parallel programming', *International Journal of Parallel Programming*, 53(2), 16. doi: 10.1007/s10766-025-00788-1. [Journal - JCR Impact Factor 1.5 (2024), Q3]
- Sabir, O. and Alebrahim, R. (2025) 'Coarse-grained column agglomeration parallel algorithm for LU factorization using multi-threaded MATLAB', *Mathematics*, 13(2), 298. doi: 10.3390/math13020298. [Journal - JCR Impact Factor 2.3 (2024), Q1]
- Thomas, A. and Kumar, A. (2018) 'A comparative evaluation of systems for scalable linear algebra-based analytics', *Proceedings of the VLDB Endowment*, 11(13), pp. 2168-2182. doi: 10.14778/3275366.3284963. [Conference - CORE Ranking A* (ICORE2026, VLDB)]